

Curso <i>Engenharia de Software</i>	Campus <i>Rio Verde</i>
<i>Programação Orientada a Objetos</i>	
Nome do(a) acadêmico(a)	Valor 10,0
Professor <i>Sergio Souza Novak</i>	Turma <i>B</i>
Data da Atividade <i>/ /2026</i>	Código <i>ESW448</i>

ATENÇÃO: Esta atividade tem como objetivo a fixação e revisão dos conceitos de OO.

ORIENTAÇÕES PARA A RESOLUÇÃO

- Responda às questões com clareza;
- Desenvolva a interpretação crítica do código em Java;

QUESTÕES

1. **Conceitos Fundamentais:** A partir dos textos de apoio abaixo:

Um paradigma de programação pode ser entendido como um *modelo* ou estilo que orienta a forma como programas de computador são concebidos e escritos. Em outras palavras, trata-se de um conjunto de conceitos, regras e práticas que guia o programador na maneira de estruturar o código, descrever as soluções e organizar os dados e as operações em um software. Paradigmas diferentes, como o imperativo, o funcional ou o orientado a objetos, oferecem visões distintas sobre como representar e resolver problemas por meio de algoritmos e linguagens de programação.

Fonte: Wikipédia. *Paradigma de programação* (pt.wikipedia.org/wiki/Paradigma_de_programação).



Com base nos seus conhecimentos sobre Programação Orientada a Objetos, explique com suas palavras o que é o paradigma de Orientação a Objetos e o que significa o conceito de Encapsulamento.

2. **Construtores:** Qual é o principal objetivo da utilização de um método construtor em uma classe na Programação Orientada a Objetos?

- A) Destruir o objeto da memória quando ele não é mais utilizado pelo sistema.
- B) Transformar a instância do objeto em uma representação textual.
- C) Inicializar os atributos do objeto com valores consistentes no momento de sua instanci-
ção.
- D) Proteger os atributos de serem acessados por outras classes do sistema.
- E) Permitir que a classe seja herdada por múltiplas subclasses.

3. **Encapsulamento e Prática:** Observe o diagrama da classe **Produto** abaixo. Escreva o código Java para implementar esta classe, garantindo que o atributo **preco** seja inacessível diretamente de fora da classe (privado) e que existam métodos para obter o preço e aplicar um desconto. Desenvolva também um construtor apropriado.

Produto
- nome : String - preco : double + Produto(nome: String, preco: double) + getPreco() : double + aplicarDesconto(percentual: double) : void

-
4. Em programação orientada a objetos, métodos de acesso do tipo *getter* (métodos de consulta) têm como principal responsabilidade:
- A) retornar o valor atualmente armazenado em um atributo encapsulado.
 - B) alterar o estado interno de um objeto por meio de regras de negócio.
 - C) garantir que o objeto instanciado seja deletado da memória.
 - D) conectar a instância do objeto a uma base de dados relacional.
 - E) construir uma nova instância da classe em memória, semelhante ao construtor.
5. O encapsulamento em Orientação a Objetos é um dos pilares fundamentais. Sobre os modificadores de visibilidade (público, privado, protegido) na maioria das linguagens OO, indique a alternativa **correta**:
- A) Um atributo público só é acessível na classe onde foi declarado.
 - B) Um método protegido é visível dentro da própria classe, nas suas subclasses e geralmente no mesmo pacote.
 - C) Um atributo privado pode ser acessado diretamente pelas subclasses, visando facilitar a herança.

- D) Membros privados podem ser modificados deliberadamente por qualquer classe que instancie o objeto, desde que autorizados.
- E) Um método público não consegue acessar métodos ou atributos privados da sua própria classe em nenhuma hipótese.

6. **Agricultura 4.0 — Estufa Inteligente:** A modelagem da classe EstufaInteligente controla variáveis ambientais para otimizar o plantio, em substituição direta a fornos industriais.

EstufaInteligente
- umidadeAtual : double - umidadeMinima : double - umidadeMaxima : double - sistemaAtivo : boolean - aguaConsumida : double + EstufaInteligente(uMin: double, uMax: double) + ligar() : void + desligar() : void + irrigar(litros: double) : double + colherFrutos() : double - umidadeIdeal() : boolean - verificarSeguranca() : void

```
01: // EstufaInteligente.java
02: public class EstufaInteligente {
03:     private double umidadeAtual;
04:     private double umidadeMinima;
05:     private double umidadeMaxima;
06:     private boolean sistemaAtivo;
07:     private double aguaConsumida;
08:
09:     public EstufaInteligente(double uMin, double uMax) {
10:         this.umidadeMinima = uMin;
11:         this.umidadeMaxima = uMax;
12:         this.umidadeAtual = 40.0;
13:         this.sistemaAtivo = false;
14:     }
15:
16:     public void ligar() { this.sistemaAtivo = true; }
17:     public void desligar() { this.sistemaAtivo = false; }
18:
19:     public double irrigar(double litros) {
20:         this.umidadeAtual += litros * 5;
21:         this.aguaConsumida += litros;
22:         verificarSeguranca();
23:         return this.umidadeAtual;
24:     }
25:
26:     public double colherFrutos() {
27:         if (sistemaAtivo && umidadeIdeal()) {
28:             return 50.0; // Producao em kg
29:         }
30:     }
31: }
```

```

30:         return 0.0;
31:     }
32:
33:     private boolean umidadeIdeal() {
34:         return umidadeAtual >= umidadeMinima && umidadeAtual
35:         <= umidadeMaxima;
36:     }
37:     private void verificarSeguranca() {
38:         if (umidadeAtual > umidadeMaxima) {
39:             desligar();
40:         }
41:     }
42: }

```

Considerando o código da classe `EstufaInteligente`, julgue as afirmações como Verdadeiras (V) ou Falsas (F) e assinale a sequência **correta**:

- () O método `umidadeIdeal` (linhas 33-35) é privado, o que significa que ele só pode ser invocado internamente pelos métodos da própria classe.
- () Na chamada do método `irrigar`, a verificação de segurança é sempre ativada internamente e garante o desligamento automático do equipamento caso a umidade supere o limite máximo permitido.
- () O processo de colher frutos (linha 26) acontece independentemente de o sistema da estufa estar ligado ou desligado.
- () É plenamente possível que uma classe em outro pacote altere de forma direta e irrestrita o valor do atributo `aguaConsumida`.

- A) V - V - F - F.
- B) V - F - V - V.
- C) F - V - V - F.
- D) V - V - V - F.
- E) F - F - F - V.

7. Assinale a alternativa que apresenta de forma correta o modificador de acesso (ou a falta dele) em Java que permite que um membro de uma classe seja acessado **apenas** por classes que pertençam ao mesmo pacote (modificador default):

- A) `private`
- B) `protected`
- C) `package-private` (omissão de modificador)
- D) `public`
- E) `static default`

8. **Modificador `protected`:** Considerando os conceitos de orientação a objetos em Java, assinale a alternativa que descreve com sucesso quem possui permissão padrão para acessar o atributo `codigoAcesso` declarado no código abaixo:

```
package sistema.seguranca;
public class Catraca {
    protected int codigoAcesso;
}
```

- A) Somente classes filhas de Catraca, independentemente do pacote, sem acesso para outras classes do mesmo pacote.
- B) Classes que estejam no mesmo pacote (sistema.seguranca) de Catraca ou classes filhas (subclasses), mesmo que estejam em outros pacotes.
- C) Qualquer classe, em qualquer pacote, desde que devidamente instanciada na memória principal.
- D) Exclusivamente os métodos que pertencem internamente à declaração da classe Catraca.
- E) Apenas as classes filhas que estiverem rigorosamente contidas dentro do mesmo pacote (sistema.seguranca).

9. Leia o texto de apoio e responda o que se pede:

Um *ensemble*, no contexto de aprendizado de máquina e inteligência artificial, é uma técnica que combina múltiplos modelos preditivos (como redes neurais, árvores de decisão ou SVMs) com o objetivo de obter uma performance superior à de um único modelo individual. A ideia central é explorar a diversidade entre os modelos, pois cada um pode capturar aspectos diferentes dos dados. Dessa forma, é possível reduzir erros sistemáticos e aumentar a robustez geral das previsões por meio de métodos como votação majoritária (para tarefas de classificação) ou média ponderada (para problemas de regressão).

Entre as principais abordagens de *ensemble learning* destacam-se três técnicas. A primeira é o *bagging* (*Bootstrap Aggregating*), utilizado por algoritmos como o *Random Forest*, que consiste em treinar diversos modelos independentes utilizando subconjuntos aleatórios dos dados. A segunda é o *boosting*, presente em algoritmos como *AdaBoost* e *XGBoost*, no qual os modelos são treinados de forma sequencial, dando maior atenção aos exemplos que foram classificados incorretamente pelos modelos anteriores. Por fim, há o *stacking*, que utiliza um modelo adicional, chamado de meta-modelo, responsável por combinar as previsões produzidas pelos modelos base.

Fonte: Wikipédia. *Ensemble learning*. Disponível em: https://pt.wikipedia.org/wiki/Ensemble_learning.

Com base nas informações do texto e nos seus conhecimentos sobre Programação Orientada a Objetos, analise o código Java abaixo que propõe uma modelagem simplificada de uma técnica de *ensemble*:

```
public class Ensemble {
    private String tipoAbordagem;
    private int quantidadeModelosBase;
    private double acuraciaGeral;

    public Ensemble(String tipo, int qtdeModelos) {
        this.tipoAbordagem = tipo;
        this.quantidadeModelosBase = qtdeModelos;
        this.acuraciaGeral = 0.0;
    }
}
```

```

public void registrarAcuracia(double novaAcuracia) {
    if (novaAcuracia >= 0.0 && novaAcuracia <= 100.0) {
        this.acuraciaGeral = novaAcuracia;
    }
}

public double getAcuraciaGeral() {
    return this.acuraciaGeral;
}
}

```

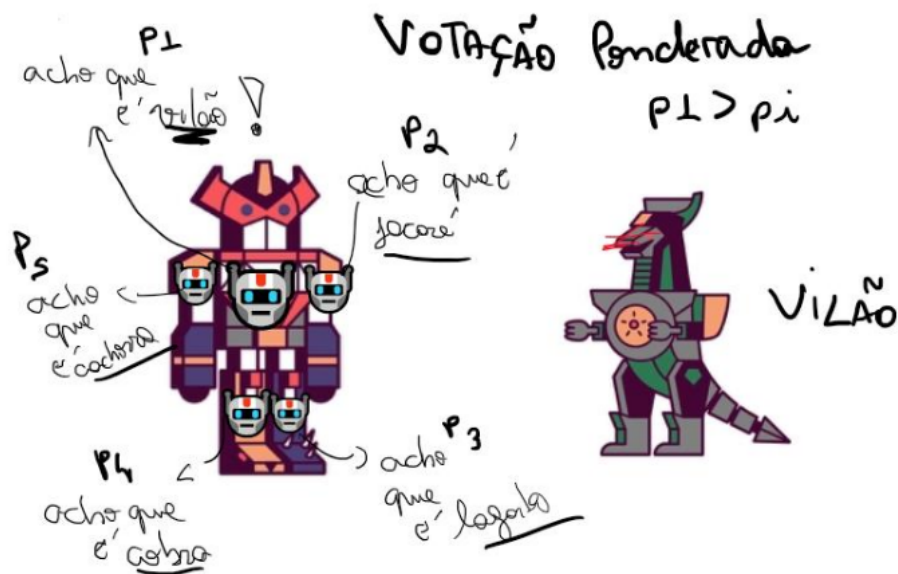


Figura 1: Fonte: Cadernos de desenho do professor Sergio. O ensemble é como se fosse um megazord da Inteligência Artificial.

Considerando as boas práticas arquiteturais e regras de visibilidade (encapsulamento), assinale a alternativa **correta**:

- A) Os atributos `tipoAbordagem` e `quantidadeModelosBase` deveriam ter a visibilidade `public` para facilitar suas alterações irrestritas durante a fase de acréscimo de novos sub-modelos.
 - B) A alteração direta e externa do atributo usando `meuEnsemble.acuraciaGeral = -10.5;` é plenamente permitida, desde que a linguagem identifique que seja a classe principal do sistema.
 - C) O método `registrarAcuracia()` permite a alteração de estado da variável sem aplicar nenhum filtro prático limitante de validade para o valor de `acuraciaGeral`.
 - D) O método construtor inicializa a variável `acuraciaGeral` com `0.0` toda vez que um novo objeto `Ensemble` é recém-instanciado na memória.
 - E) Qualquer classe, em qualquer pacote, tem total acesso ao método `getAcuraciaGeral()`, porém é impedida de examinar o número devolvido visto que a variável que o preenche é declarada como `private`.
10. No contexto de IoT nas cidades e campos modernos, modelou-se uma classe representante de um VANT (Veículo Aéreo Não Tripulado).

DronePulverizador
- modelo : String - bateriaAtual : double - capacidadePesticida : double - nivelPesticida : double - emVoo : boolean + DronePulverizador(modelo: String, cap: double) + decolar() : void + pousar() : void + pulverizar(area: double) : boolean - consumirBateria(area: double) : void

```

01: public class DronePulverizador {
02:     private String modelo;
03:     private double bateriaAtual;
04:     private double capacidadePesticida;
05:     private double nivelPesticida;
06:     private boolean emVoo;
07:
08:     public DronePulverizador(String mod, double cap) {
09:         this.modelo = mod;
10:         this.capacidadePesticida = cap;
11:         this.nivelPesticida = cap; // tanque abastecido
12:         this.bateriaAtual = 100.0;
13:         this.emVoo = false;
14:     }
15:
16:     public void decolar() {
17:         if(this.bateriaAtual > 20.0) { this.emVoo = true; }
18:     }
19:     public void pousar() { this.emVoo = false; }
20:
21:     public boolean pulverizar(double area) {
22:         if (!this.emVoo) { return false; }
23:         double gasto = area * 0.5;
24:         if (this.nivelPesticida >= gasto) {
25:             this.nivelPesticida -= gasto;
26:             consumirBateria(area);
27:             return true;
28:         }
29:         return false;
30:     }
31:
32:     private void consumirBateria(double area) {
33:         this.bateriaAtual -= area * 1.5;
34:         if (this.bateriaAtual <= 20.0) {
35:             this.pousar(); // pouso de emergencia provocado
36:         }
37:     }
38: }

```

Com base no código da classe DronePulverizador, analise as afirmações abaixo e assinale a alternativa que apresenta a sequência **correta** respectiva (V para Verdadeiro e F para Falso):

- () O método `consumirBateria` possui visibilidade privada (linha 32), impedindo sua chamada direta externa por outras classes e sendo o responsável, se necessário, por disparar um pouso forçado de emergência.
- () O drone sempre decola (muda o estado `emVoo` para `true`) cada vez que o método `decolar` é chamado no sistema principal, não importando como está seu nível de bateria naquele momento.
- () O drone em momento algum injeta defensivo se não estiver garantidamente em estado de voo ativo e não possuir pesticida em quantidade suficiente no seu tanque.
- () A leitura direta da `bateriaAtual` bem como sua alteração arbitrária por qualquer classe em outro arquivo é aceitável, visto que o atributo é deixado visível.

- A) V - F - V - F.
- B) F - V - F - V.
- C) V - V - F - F.
- D) F - F - V - V.
- E) V - F - F - V.

11. Considere a modelagem de uma classe `Termostato` (usada em aparelhos de Ar Condicionado Inteligentes) que possui um atributo restrito e fortemente encapsulado, denominado `temperaturaDesejada`

Termostato	
-	<code>temperaturaDesejada : double</code>
+	<code>getTemperatura() : double</code>
+	<code>setTemperatura(temp: double) : void</code>

Explique, conceitualmente, a extrema importância de se utilizar abordagens controladas via métodos *getters* e *setters* em vez de permitir o acesso e intervenção direta no atributo `temperaturaDesejada` (**limite-se apenas à linguagem humana verbal escrita, sem programação**). Complementarmente, dê um bom exemplo lúdico de regra de negócio/controle de limites que faria todo sentido e poderia ser codificada no corpo do método `setTemperatura(temp: double)`, evitando falhas do dispositivo de refrigeração.

12. Leia o texto e responda o que se pede:

O carrinho de compras na internet é uma funcionalidade essencial dos sites de comércio eletrônico, responsável por reunir de forma organizada os itens selecionados pelo consumidor ao longo da navegação. Ele funciona como um espaço virtual onde os produtos escolhidos são armazenados temporariamente, permitindo adicionar, remover ou alterar quantidades antes da finalização do pedido, espelhando o comportamento do carrinho físico em uma loja tradicional.

Fonte: *Shopify. Carrinho de compras online* (www.shopify.com/br/blog/carrinho-de-compras-online).

Analisar o trecho de código Java abaixo, que modela um CarrinhoDeCompras.

```
public class CarrinhoDeCompras {  
    private double totalCompras;  
  
    public CarrinhoDeCompras() {  
        this.totalCompras = 0.0;  
    }  
  
    public void adicionarItem(double preco) {  
        if (preco > 0) {  
            this.totalCompras += preco;  
        }  
    }  
  
    // Metodo Y
```

}

Qual das opções abaixo representa o código correto para o "Metodo Y", de forma a respeitar o encapsulamento, permitindo que o valor total seja lido, porém sem permitir que o valor do atributo totalCompras seja modificado diretamente de fora da classe?

- A) `public void setTotalCompras(double total) { this.totalCompras = total; }`
- B) `public double getTotalCompras() { return this.totalCompras; }`
- C) `private double lerTotal() { return totalCompras; }`
- D) `public double getAndSetTotal(double total) { this.totalCompras = total; return total; }`
- E) `protected void exibirTotal() { System.out.println(totalCompras); }`

13. Considere a classe BateriaSmartphone descrita a seguir e marque Verdadeiro (V) ou Falso (F) para as afirmações sobre a sua estrutura.

```
public class BateriaSmartphone {  
    private int carga;  
  
    public BateriaSmartphone() {  
        this.carga = 100;  
    }  
  
    public void usarBateria(int consumo) {  
        if (this.carga - consumo >= 0) {  
            this.carga -= consumo;  
        } else {  
            this.carga = 0;  
        }  
    }  
  
    public void carregarBateria(int quantidade) {  
        this.carga += quantidade;  
        if (this.carga > 100) {  
            this.carga = 100;  
        }  
    }  
  
    public int getCarga() {  
        return this.carga;  
    }  
}
```

- () A classe não protege adequadamente o atributo carga, pois qualquer outra classe pode facilmente instanciar o objeto e usar `bateria.carga = 200;`.
- () Os métodos `usarBateria` e `carregarBateria` garantem a integridade dos dados e impõem limites de regra de negócio, impedindo a carga de ficar negativa ou superior a 100.
- () O método construtor garante que, toda vez que um novo objeto `BateriaSmartphone` for instanciado na memória, ele inicie sempre com sua carga em 100.
- () O único método através do qual agentes externos a esta classe conseguem saber a quantidade exata de carga de energia restante é utilizando o `getCarga()`.

- A) V - V - V - F.
- B) F - V - V - V.
- C) V - F - V - V.
- D) F - F - V - V.
- E) F - V - F - V.

14. **Prática de Código (Discursiva):** A classe abaixo representa um Funcionario com salário encapsulado. No entanto, ela necessita de um método que conceda um aumento ao funcionário com base em uma porcentagem (ex: 10%).

```
public class Funcionario {  
    private String nome;  
    private double salario;  
  
    public Funcionario(String nome, double salario) {  
        this.nome = nome;  
        this.salario = salario;  
    }  
  
    public double getSalario() {  
        return this.salario;  
    }  
  
    // Escreva o metodo de aumento aqui  
}
```

Escreva, em sintaxe Java, o método `aplicarAumento(double porcentagem)` que deve ser inserido na classe. O método não deve retornar nada (`void`), mas deve alterar o valor do atributo `salario` somando o correspondente da porcentagem passada como parâmetro. Adicione também uma verificação simples: apenas conceda o aumento se a porcentagem for maior que zero.

15. Leia o texto de apoio e responda o que se pede:

Um *overfitting* ocorre quando um modelo de aprendizado de máquina aprende os dados de treinamento com excesso de detalhe, capturando não apenas os padrões gerais, mas também ruídos e peculiaridades específicas do conjunto de treino. Isso faz com que o modelo apresente excelente performance nos dados de treinamento, mas generalize mal para novos dados não vistos, resultando em alta variância e baixa capacidade de predição em situações reais.

Por outro lado, o *underfitting* acontece quando o modelo é incapaz de capturar adequadamente os padrões subjacentes dos dados, apresentando performance ruim tanto no conjunto de treinamento quanto nos dados de teste. Esse problema está geralmente associado a modelos excessivamente simples ou com tempo de treinamento insuficiente, caracterizando-se por alta viés e incapacidade de aprender a complexidade inerente aos dados.

Fonte: Wikipédia. *Overfitting e Underfitting*. Disponível em: <https://pt.wikipedia.org/wiki/Overfitting>.

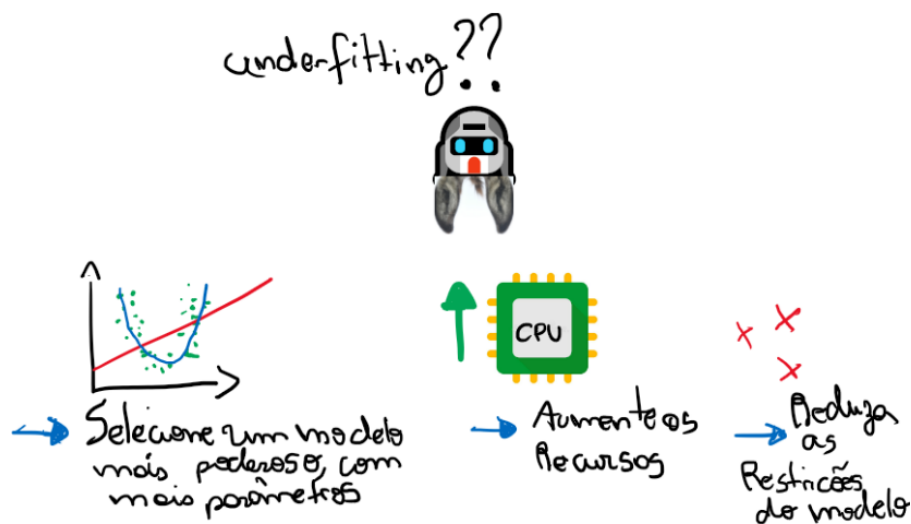


Figura 2: Ilustração de Underfitting.

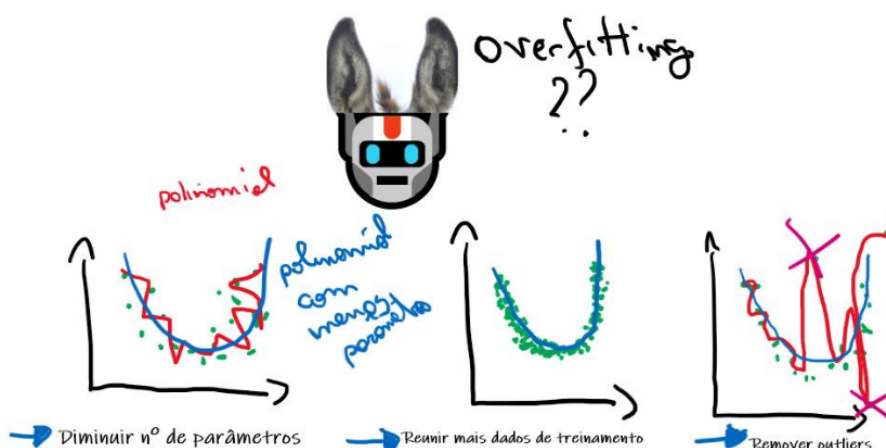


Figura 3: Ilustração de Overfitting.

Fonte: Cadernos de desenho do professor Sergio.

Com base nas ilustrações acima e nos conceitos de Programação Orientada a Objetos, analise a implementação da classe `ModeloIA` abaixo:

```

public class ModeloIA {
    private String nomeAlgoritmo;
    private int grauPolinomio;
    private double taxaAprendizado;
    private double regularizacao;
    private int epocasTreinamento;

    public ModeloIA(String nome, int grau, double taxa) {
        this.nomeAlgoritmo = 101; // Erro 1: Atribuindo int a
        String
        setGrauPolinomio(grau);
        setTaxaAprendizado(taxa);
        this.regularizacao = 0.01;
        this.epocasTreinamento = 500.5; // Erro 2: Atribuindo
        double a int
    }

    public void setGrauPolinomio(int grau) {
        if (grau >= 1 && grau <= 10) {
            this.grauPolinomio = grau;
        } else {
            System.out.println("Falha: Grau deve estar entre 1 e
            10.");
        }
    }

    public void setTaxaAprendizado(double taxa) {
        if (taxa > 0.0 && taxa < 1.0) {
            this.taxaAprendizado = taxa;
        }
    }

    public void ajustarRegularizacao(double reg) {
        if (reg >= 0.0 && reg <= 0.5) {
            this.regularizacao = reg;
        }
    }

    public void treinar() {
        if (grauPolinomio > 0 && taxaAprendizado > 0) {
            System.out.println("Treinando " + nomeAlgoritmo + "...
            ");
        }
    }

    public double getTaxaAprendizado() { return taxaAprendizado; }
}

```

Considere a implementação da classe ModeloIA apresentada e os princípios da linguagem Java. Analise a existência de erros de compilação e assinale a alternativa que apresenta a contagem e a justificativa **corretas**:

- A) Não há erros de compilação, pois o Java realiza a conversão automática (casting) de valores numéricos para String e de double para int durante a atribuição direta.

- B) Existem dois erros de compilação: um na atribuição do valor 101 ao atributo nomeAlgoritmo (incompatibilidade entre int e String) e outro na atribuição de 500.5 ao atributo epocasTreinament (incompatibilidade entre double e int).
- C) Existe apenas um erro de compilação, localizado no método treinar(), pois não é permitido concatenar uma variável do tipo String em um comando de impressão System.out.println.
- D) Existe apenas um erro de compilação na linha do construtor, pois o Java não permite invocar métodos setters (como setGrauPolinomio) dentro de um método construtor da própria classe.
- E) Existem dois erros de compilação: ambos referentes ao uso do modificador private, que impede que os próprios métodos da classe (como o construtor) acessem seus atributos internos.